

iResolve — Module 1: Ad-hoc Letter Request

Approvals & Reconciliation Workflow — Idea / Spec Note

1. Problem Statement

Module 1 generates eligible accounts/counts against a set of criteria and exports an Excel file. The ops team reviews this file and may remove accounts before approving; a super admin then downloads the file and triggers the actual letters. Batches range from roughly 30,000 to 200,000 accounts.

Current gap:

- If ops edits the exported Excel offline, there is no reliable way to detect exactly what changed before letters go out.
- Emailing the edited file back is not viable: the data is real customer PII, and files regularly exceed 35 MB — too large for most mail systems, and email is a weak transport for PHI/PII regardless of size.
- The current process has no enforced maker-checker separation and no built-in audit trail of who removed which accounts, or why.

2. Goals

- Eliminate email as a transport for the eligibility file entirely — everything stays inside iResolve.
- Give the system a reliable, automatic way to detect and display exactly what ops changed (reconciliation/diff), instead of trusting the file blindly.
- Introduce a formal maker-checker approval chain: Ops → HOD → Programming (trigger), with Super Admin retaining full override access.
- Keep every state transition, comment, and approval attributable and auditable.
- Design for scale from day one (30k–200k accounts per batch) — no UI or parsing approach that only works at small volumes.

3. Proposed Solution — "Approvals" Page

A new page inside Module 1 lists every run/batch with its counts, current status, and actions. Each row supports:

- **Download** — original eligibility export for that run.
- **Re-upload** — ops submits their edited version through the app (authenticated, streamed upload — not email).
- **Open** — a detail view showing the full diff: system-level rejection reasons from the original run, plus the specific accounts ops removed.
- **Approve** — role-gated action buttons for Ops and HOD respectively, each enforced server-side.

3.1 Reconciliation logic

On re-upload, the system reads the account IDs present in the returned file and diffs them against the original batch's account set (a straightforward, indexed set-difference — fast even at 200k rows). This produces an automatic **edited count** with no manual reconciliation needed. The diff is computed server-side; the uploaded file itself is never treated as authoritative for anything beyond "which account IDs remain." Letter content/data is always pulled fresh from the database by account ID, not from the spreadsheet's own fields.

3.2 Two reason sources — kept separate

- **System rejection reasons** — generated by the eligibility engine at run time (e.g. bankruptcy flag, litigation hold, insufficient balance). Fixed once the run happens; shown as read-only context in the Open detail view.
- **Ops edit comment** — a single mandatory free-text comment tied to the upload event itself ("reason for the edited count"), not attached to individual accounts. The Approve button for Ops stays disabled — and the API rejects the call — until this comment is present.

4. Approval Workflow (state machine)

Status	Trigger	Actor
GENERATED	Eligibility run completes; Excel exported	System
RECONCILED	Ops re-uploads edited Excel; diff computed	Ops
OPS_APPROVED	Ops enters mandatory comment and approves	Ops
TRIGGER_READY	HOD reviews diff + comment and approves	HOD
TRIGGERED	Letters generated/sent	Programming team
REJECTED / SENT_BACK	Ops or HOD sends batch back for re-edit, with reason	Ops / HOD

5. Roles & Permissions

- **Ops** — can download, re-upload, view the diff, and approve only after entering the mandatory edit comment.
- **HOD** — a dedicated permission (e.g. `approve_batch_hod`) required to unlock the HOD-approve action. This must be enforced in the API endpoint itself via the existing `require_module()` dependency — a disabled button in the UI is a courtesy only, not a control.
- **Programming team** — read access to `TRIGGER_READY` batches and the ability to download/trigger; no approval rights.
- **Super Admin** — full access/bypass on every step, but actions taken via bypass are flagged (`acted_as_superadmin: true`) in the audit log so bypasses remain distinguishable from normal HOD approvals.

6. Data Model (sketch)

batch — one row per eligibility run

```
batch_id • run_date • criteria_snapshot • original_count • current_count • status • created_by
```

run_exclusions — system-level rejections at generation time

```
batch_id · account_id · reason_code · reason_text · created_at
```

batch_diff — accounts removed by ops on re-upload

```
batch_id · account_id · source ('ops_upload') · created_at
```

batch_approval_log — full audit trail of the approval chain

```
batch_id · actor_id · role ('ops' | 'hod' | 'superadmin' | 'programming') · action ('upload' |  
'approve' | 'reject' | 'trigger') · comment · created_at
```

The "Open" detail view for a run queries all three of `run_exclusions`, `batch_diff`, and `batch_approval_log` by `batch_id` and renders them as distinct sections: system rejections, ops-removed accounts, and the approval timeline.

7. Scale Considerations (30k–200k accounts/batch)

- Review/diff UI uses server-side pagination and virtualized scrolling — never render the full account list client-side.
- Excel export/import uses streaming read/write (e.g. `openpyxl` read-only/write-only mode) rather than loading the full workbook into memory.
- Reconciliation runs as a background job (Celery), not inline on the upload request; the account-ID diff itself is a simple indexed set-difference and is fast even at the upper end of this range.
- The Open/diff view paginates the removed-accounts list rather than rendering all rows at once, with a downloadable audit CSV for anyone who needs the full list offline.
- Upload endpoint streams to disk and has body-size limits raised accordingly (35 MB+ files are normal at this volume).

8. Security Notes

- No PII ever leaves the platform via email or external transport under this design.
- Every approval action is authenticated via the existing ADFS SSO / Windows Auth and logged with actor, role, and timestamp.
- Re-uploaded files are validated server-side (expected account IDs only, no unexpected additions) before being accepted into the reconciliation flow.
- Letter content is always sourced from the database by account ID — the uploaded spreadsheet is used only to determine inclusion/exclusion, never trusted for other field values.